

1

让自己习惯 C++

Accustoming Yourself to C++

不论你的编程背景是什么，C++ 都可能让你觉得有点儿熟悉。它是一个威力强大的语言，带着众多特性，但是在你可以驾驭其威力并有效运用其特性之前，你必须先习惯 C++ 的办事方式。本书谈的便是这个。总有某些东西比其他更基础些，本章就是最基本的一些东西。

条款 01: 视 C++ 为一个语言联邦

View C++ as a federation of languages.

一开始，C++ 只是 C 加上一些面向对象特性。C++ 最初的名称 **C with Classes** 也反映了这个血缘关系。

但是当这个语言逐渐成熟，它变得更活跃更无拘束，更大胆更冒险，开始接受不同于 **C with Classes** 的各种观念、特性和编程战略。**Exceptions**（异常）对函数的结构化带来不同的做法（见条款 29），**templates**（模板）将我们带到新的设计思考方式（见条款 41），**STL** 则定义了一个前所未有的伸展性做法。

今天的 C++ 已经是个多重范型编程语言（**multiparadigm programming language**），一个同时支持过程形式（**procedural**）、面向对象形式（**object-oriented**）、函数形式（**functional**）、泛型形式（**generic**）、元编程形式（**metaprogramming**）的语言。这些能力和弹性使 C++ 成为一个无可匹敌的工具，但也可能引发某些迷惑：所有“适当用法”似乎都有例外。我们该如何理解这样一个语言呢？

最简单的方法是将 C++ 视为一个由相关语言组成的联邦而非单一语言。在其某个次语言（**sublanguage**）中，各种守则与通例都倾向简单、直观易懂、并且容易

记住。然而当你从一个次语言移往另一个次语言，守则可能改变。为了解 C++，你必须认识其主要的次语言。幸运的是总共只有四个：

- **C**。说到底 C++ 仍是以 C 为基础。区块 (blocks)、语句 (statements)、预处理器 (preprocessor)、内置数据类型 (built-in data types)、数组 (arrays)、指针 (pointers) 等统统来自 C。许多时候 C++ 对问题的解法其实不过就是较高级的 C 解法 (例如条款 2 谈到预处理器之外的另一选择，条款 13 谈到以对象管理资源)，但当你以 C++ 内的 C 成分工作时，高效编程守则映照出 C 语言的局限：没有模板 (templates)，没有异常 (exceptions)，没有重载 (overloading)……
- **Object-Oriented C++**。这部分也就是 C with Classes 所诉求的：classes (包括构造函数和析构函数)，封装 (encapsulation)、继承 (inheritance)、多态 (polymorphism)、virtual 函数 (动态绑定)……等等。这一部分是面向对象设计之古典守则则在 C++ 上的最直接实施。
- **Template C++**。这是 C++ 的泛型编程 (generic programming) 部分，也是大多数程序员经验最少的部分。Template 相关考虑与设计已经弥漫整个 C++，良好编程守则中“惟 template 适用”的特殊条款并不罕见 (例如条款 46 谈到调用 template functions 时如何协助类型转换)。实际上由于 templates 威力强大，它们带来崭新的编程范型 (programming paradigm)，也就是所谓的 template metaprogramming (TMP，模板元编程)。条款 48 对此提供了一份概述，但除非你是 template 激进团队的中坚骨干，大可不必太担心这些。TMP 相关规则很少与 C++ 主流编程互相影响。
- **STL**。STL 是个 template 程序库，看名称也知道，但它是非常特殊的一个。它对容器 (containers)、迭代器 (iterators)、算法 (algorithms) 以及函数对象 (function objects) 的规约有极佳的紧密配合与协调，然而 templates 及程序库也可以其他想法建置出来。STL 有自己特殊的办事方式，当你伙同 STL 一起工作，你必须遵守它的规约。

记住这四个次语言，当你从某个次语言切换到另一个，导致高效编程守则要求你改变策略时，不要感到惊讶。例如对内置 (也就是 C-like) 类型而言 *pass-by-value* 通常比 *pass-by-reference* 高效，但当你从 C part of C++ 移往 Object-Oriented C++，由于用户自定义 (user-defined) 构造函数和析构函数的存在，*pass-by-reference-to-const* 往往更好。运用 Template C++ 时尤其如此，因为彼时你

甚至不知道所处理的对象的类型。然而一旦跨入 STL 你就会了解, 迭代器和函数对象都是在 C 指针之上塑造出来的, 所以对 STL 的迭代器和函数对象而言, 旧式的 C *pass-by-value* 守则再次适用 (参数传递方式的选择细节请见条款 20)。

因此我说, C++ 并不是一个带有一组守则的一体语言; 它是从四个次语言组成的联邦政府, 每个次语言都有自己的规约。记住这四个次语言你就会发现 C++ 容易了解得多。

请记住

- C++ 高效编程守则视状况而变化, 取决于你使用 C++ 的哪一部分。

条款 02: 尽量以 const, enum, inline 替换 #define

Prefer consts, enums, and inlines to #defines.

这个条款或许改为“宁可以编译器替换预处理器”比较好, 因为或许 #define 不被视为语言的一部分。那正是它的问题所在。当你做出这样的事情:

```
#define ASPECT_RATIO 1.653
```

记号名称 ASPECT_RATIO 也许从未被编译器看见; 也许在编译器开始处理源码之前它就被预处理器移走了。于是记号名称 ASPECT_RATIO 有可能没进入记号表 (symbol table) 内。于是当你运用此常量但获得一个编译错误信息时, 可能会带来困惑, 因为这个错误信息也许会提到 1.653 而不是 ASPECT_RATIO。如果 ASPECT_RATIO 被定义在一个非你所写的头文件内, 你肯定对 1.653 以及它来自何处毫无概念, 于是你将因为追踪它而浪费时间。这个问题也可能出现在记号式调试器 (symbolic debugger) 中, 原因相同: 你所使用的名称可能并未进入记号表 (symbol table)。

解决之道是以一个常量替换上述的宏 (#define):

```
const double AspectRatio = 1.653;    //大写名称通常用于宏,  
                                     //因此这里改变名称写法。
```

作为一个语言常量, AspectRatio 肯定会被编译器看到, 当然就会进入记号表内。此外对浮点常量 (floating point constant, 就像本例) 而言, 使用常量可能比使用 #define 导致较小量的码, 因为预处理器“盲目地将宏名称 ASPECT_RATIO 替换为 1.653”可能导致目标码 (object code) 出现多份 1.653, 若改用常量 AspectRatio 绝不会出现相同情况。